CISC 322
Assignment 3

Architectural Enhancement Proposal of Gemini CLI

April 6, 2026

The Unemployed
Gabrielle Garey (22dny@queensu.ca)
Kesiya Jacob (22ccf5@queensu.ca)
Sofiia Bushareb (22wx56@queensu.ca)
Ryan Walsh (22bfr3@queensu.ca)
Grisha Tyukin (22ld34@queensu.ca)

1. Abstract

This report explores the addition of a session replay and recording feature to Gemini CLI, focusing on the architectural design and its implications. The proposed feature allows users to capture complete interaction histories, including user prompts, model responses, and tool calls, in a structured format that can be replayed later for debugging and comparing models. The report presents two different approaches for implementing this feature. Approach A introduces a publish-subscribe style system within the core architecture, where a central event bus sends out key events that happen during a session. These events are recorded by a SessionLogger and later used by a ReplayRunner to replay the session in a clear and accurate way. Approach B uses the existing telemetry system by adding a new component that gathers the logs and traces already being produced and organizes them into a format that can be used in replay sessions. A SAAM analysis is used to evaluate both approaches based on key stakeholders and non-functional requirements such as replay correctness and testability for developers, usability for users, and maintainability of the system. Based on this analysis, Approach A is ultimately chosen as the best approach and explored further by examining how it fits into the existing core and CLI architecture. Additionally, the report delves into the potential risks of the enhancement, use cases, and plans for testing.

2. Introduction and Overview

As part of this project, our team was tasked with proposing a new feature or enhancement that is not currently available in the system and justifying its value. In many current software development practices, AI tools like Gemini CLI are widely used to generate code, automate tasks, and help with problem-solving. However, there is currently no built-in way to go back and review past interactions or consistently evaluate how models perform over time. To address this, we introduce a session replay and recording feature for Gemini CLI, which aims to improve debugging and overall development efficiency. As developers increasingly rely on AI-assisted tools, the ability to capture and revisit full interaction sessions becomes important for understanding system behaviour and comparing model outputs.

This report provides an overview of the motivation behind the feature and how it fits within the existing system. It begins by briefly describing the current architecture of the CLI and core components, followed by a discussion of the design approaches considered. These approaches are evaluated and compared using a SAAM (Software Architecture Analysis Method) analysis to determine how well they meet stakeholder needs and system requirements. The report then explores the selected design in more detail, including its integration, potential risks, and testing strategies. It concludes with the lessons learned, data dictionary, naming conventions, and a report on the contributions of the AI teammate.

3. Enhancement Proposal

The proposed enhancement to Gemini CLI is the addition of a replay feature. The goal of the feature is to record sessions for debugging or demos, as well as allow for a user to replay these sessions to compare responses against different models.

The initial vision is that the user will type a flag into their terminal to signal the start of a session they want to record. Gemini CLI will then capture the subsequent responses of the model, tracking data such as user input, tokens used, tools requested, tools executed, time taken to respond, etc. At the end of the session, the user can send another flag to receive an overview of model performance from the data collected. They can also choose to switch to a different model and compare the performance of responses from the LLM.

3.1 Motivation

As of this report, Gemini CLI offers AI-powered capabilities such as command line prompting, code generation, and workflow automation directly within the developers terminal. While these features support rapid development and iteration, there is currently limited visibility into how model outputs evolve over time or perform across repeated tasks. Incorporating a replay feature would naturally extend Gemini CLI's existing architecture by introducing evaluative feedback and transparency into the development loop.

3.2 Value and Benefits

The feature will allow users to review prior interactions, compare outputs, and understand model behaviour across iterations. The hope is that this allows users to make more informed decisions about models and refine their prompts/ workflows, improving the quality of these ai outputs. As AI continues to be a major part of the modern developers workflow, it is important to be able to evaluate model responses, ensuring that they are reliable, consistent, and aligned with user intent.

3.3 Process

This feature is inspired by our experience using AI in our own coding projects and throughout this course. We often talk about using different AI models depending on what task we are completing, and believe that being able to quantify that discussion would be a useful tool in learning what prompts and models lead to the best results. Using the knowledge from our concrete architecture report showed us how much logging already exists in the architecture, unbeknownst to the user. This is what helped us decide on the approaches for this feature.

3.4 Approach A

Approach A is based on the publish-subscribe architecture discussed in lecture. Here, a broadcast component is added to the server layer and communicates with packages/core. Two new subscriber components are created in the services layer: SessionLogger and ReplayRunner, which respond to emitted events. The central component is the broadcast component: SessionEventBus, which defines a set of events covering every meaningful movement in the session. Meaningful events include user turns, model responses, tool requests, tool results, and approval decisions. SessionLogger is a subscriber that listens to the bus and serializes each event to a JSONND file on disk. ReplayRunner is a second subscriber and a consumer of that previously recorded file. It uses the log file to rerun the session.

In terms of how this interacts with the rest of the GeminiCLI architecture, packages/CLI remains largely untouched in the role of handling user input and rendering output, and the Gemini API integration inside packages/core also remains largely unchanged. What changes is that the orchestration layer, the part that manages the turn loop, dispatches tool calls, and tracks session state, now fires events onto the bus as a side effect of its normal operation.

SessionLogger and ReplayRunner are opt-in subscribers that only attach when the – record or –replay flags are passed, meaning the entire mechanism is invisible and zero-cost in a normal interactive session.

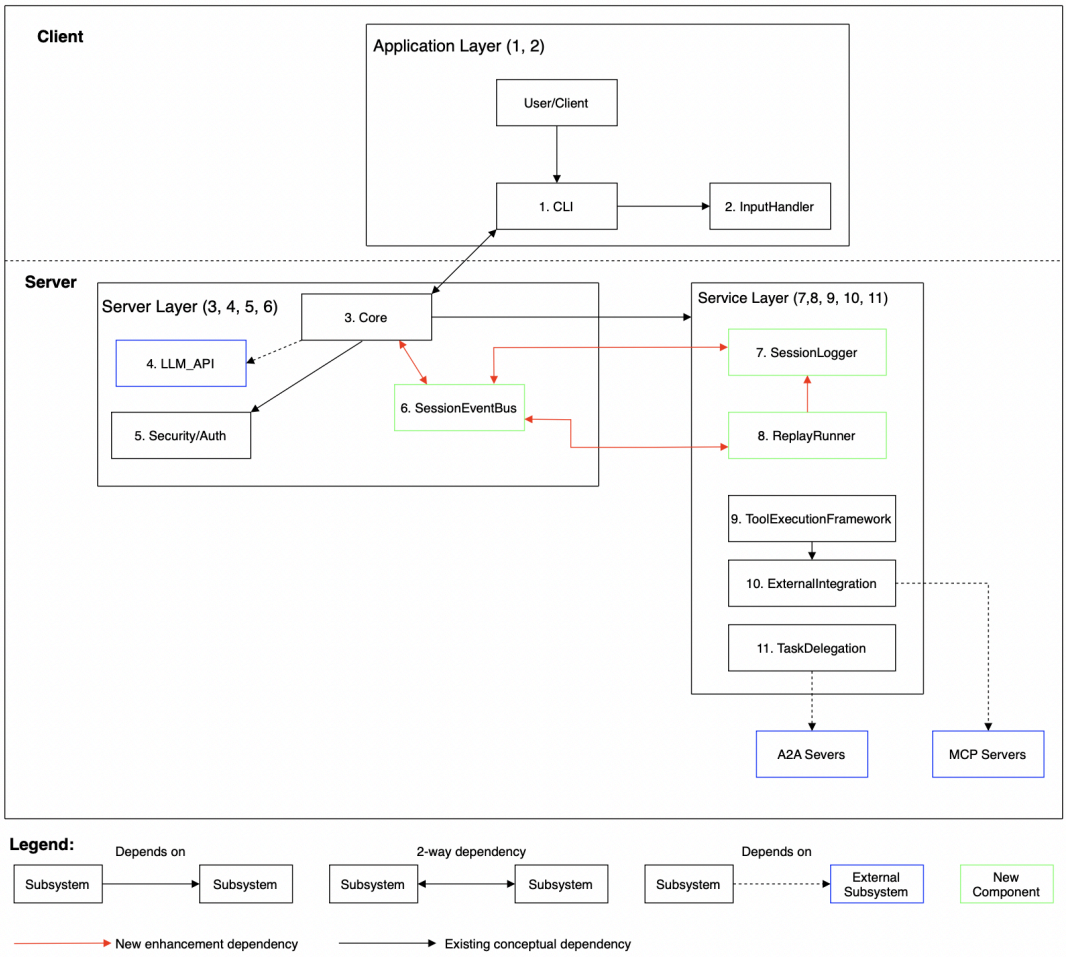

Figure 1. Approach A Implementation and Impacted Subsystem (High Level)

3.5 Approach B

The second proposed approach uses a telemetry extension architecture where the existing OpenTelemetry pipeline inside packages/core is extended with a new replay-oriented exporter, rather than introducing a separate event system. The key components are the existing telemetry layer already embedded in packages/core, which fires structured spans and logs as it orchestrates calls to the Gemini API, the auth layer, and the tool execution layer. A new ReplayExporter will sit alongside the current local and gcp exporters and log a replay-oriented view of those spans to an NDJSON file on disk. Additionally, there will be a companion ReplayRunner that consumes that file to reconstruct and re-drive the session against either the original or a substituted model. In terms of how control flows through the system, packages/cli receives the --record flag via its input handler and passes it inward to packages/core, which already handles exporter selection through its telemetry configuration in .gemini/settings.json, meaning the replay file simply

becomes another telemetry target, activated by configuration. packages/core itself changes very little, since the telemetry hooks are already wired throughout its turn loop and fire as a side effect of normal orchestration. The ReplayExporter is an observer on that existing signal. It attaches, listens, and writes, without touching the orchestration logic itself. The main caveat is that because the OTEL schema was designed for observability rather than deterministic replay, the ReplayRunner operates on best-effort data: if logPrompts is disabled for privacy reasons, prompt content will be absent from the log, and fine-grained session state may be incomplete.

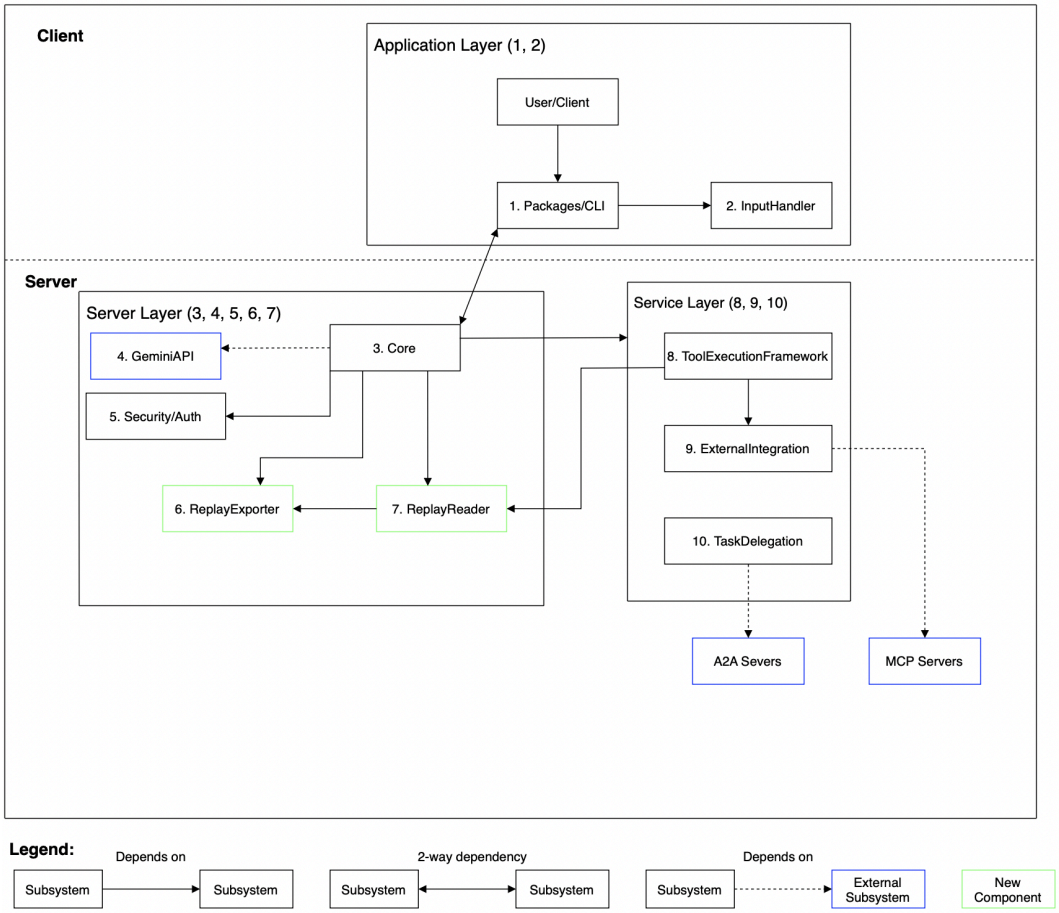

Figure 2. Approach B Implementation and Impacted Subsystems

3.5 Advantages and Disadvantages

Approach A	**Approach B**
Advantages	
The pub sup approach allows for more customization when implementing the replay feature with another model. The event schema can be customized directly, making the developer able to decide what information they want displayed with this feature instead of relying on the telemetry layer that already exists.	The event capture mechanism already exists in the architecture, it just doesn't allow the user to see it. The telemetry layer already tracks session logs, tool execution logs, and conversation end events, most of which covers most of the data backed replay needs.

The ReplayRunner can access core directly, meaning that model comparison replays go through the exact same prompt construction and tool call logic as live sessions. Output differences are a confirmed difference in AI models, not just architecture.	This approach aligns with the patterns and configurations of the system already in place. This makes the improvement much more likely to merge upstream and not uproot pre-existing dependencies. Additionally, improvements to the telemetry schema could improve replay.
Disadvantages	
Requires extensive schema designs up front. If the developer misses an event type early on, old log files won't be replayable after a schema change. This limits the maintainability of the architecture.	The logPrompts privacy setting can be disabled, meaning that prompt content may be excluded from the telemetry log. Replay without the original prompt is incomplete, so that case would need to be handled.
This implementation of the feature is a lot more work for all parties involved. Non-deterministic side effects such as shell commands, file writes, and web fetches require some sort of "mock mode" in the replay path which adds complexity.	The telemetry layer was designed for observability, not deterministic replay. It doesn't capture everything needed. For example, the exact tokens used or which branch of a conditional tool approval a user took without extending the telemetry layer in ways that don't feel natural.

4. SAAM Analysis

Major Stakeholder	**NFRs regarding enhancement for stakeholder**
Creators and maintainers of Gemini CLI: Engineers responsible for maintaining and shipping the Gemini CLI codebase, including the replay feature	*Security:* the feature must not introduce vulnerabilities into the codebase *Maintainability:* The implementation should be straightforward to understand, test, and update as the CLI evolves.
Software developers: Users who need to record sessions to diagnose unexpected model behaviour, trace tool call failures, or compare outputs across runs	*Performance:* Recording should not considerably slow down the live CLI session. *Accuracy:* The feature must capture every prompt, response and tool call without omitting or distorting anything.
Testers/researchers: Users who replay the same sessions against different AI models to compare outputs for evaluation or benchmarking	*Scalability:* The feature should handle long, multi-tool sessions without performance degradation. *Reliability/Availability*: Feature should work consistently across environments and not fail for large or complex sessions.

Casual Users: Users who want to occasionally revisit or share a session, without deep technical knowledge of the tooling.	*Portability:* Recorded sessions should be easily sharable across different machines/environments. *Usability:* The record and replay flags should be self-explanatory and require no setup beyond the flag itself.

NFR	**Approach A**	**Approach B**
Security	- Creators and maintainers *Moderate-low*. Captures full session data (prompts, tool calls, outputs), which may include sensitive information. Requires additional safeguards that can introduce risks if not handled properly.	- Creators and maintainers *High*. Builds on existing telemetry infrastructure, which already enforces secure data handling and limits unnecessary exposure
Maintain-ability	- Creators and maintainers *High*. Modular architecture (SessionEventBus, SessionLogger, ReplayRunner) separates concerns cleanly and is easy to extend or modify.	- Creators and maintainers *Moderate.* Reusing telemetry reduces new code, but tightly couples replay logic to the specific internal data structures, behaviour, and implementation details of the telemetry pipeline, making future changes harder.
Accura-cy	- Software developers *High.* Explicitly designed to capture every prompt, tool call, response precisely and in order	- Software developers *Moderate-Low.* OpenTelemetry is not meant for exact session replay. It focuses more on tracking system performance, usage patterns, errors. The events may be summarized, masked, or filtered (*Observability with OpenTelemetry*). As a result, some details needed for exact session reconstruction may be missing.
Scalabili-ty	- Testers/Researchers *Moderate-High.* NDJSON scales well because you don't need to load the entire file into memory to process it. You can read it line by line, so even very long sessions can be handled in a streaming fashion. However, very large NDJSON files still take up a lot of storage. Also, even with line by line processing, reading, searching, or replaying very large files can take noticeable time.	- Testers/Researchers *Moderate.* The telemetry pipeline can handle a lot of data efficiently for monitoring, but logs aren't structured for exact session replay. Because of this, reconstructing long or complex sessions can be slow, memory-intensive, or incomplete, making large scale replay less practical.
Reliabili-ty/Availa-bility	- Testers/Researchers *High.* The log file includes all key session data, so replay does not depend on external systems like telemetry pipelines. Because	- Testers/Researchers *Moderate.* Replay depends on telemetry being correctly enabled and configured. If any part of the telemetry pipeline fails,

	all needed information is stored in one place, the session can be replayed consistently across different environments without missing data or unexpected differences.	replay might be inconsistent or incomplete. Because it relies on multiple components, there are more potential points of failure.
Portabili-ty	- Casual users *High*. NDJSON files store all session events in a standard, readable format. They can be easily shared, transferred, or stored without requiring special tools or configurations. Because the replay feature only depends on the log file itself, sessions can be reproduced consistently across different machines and environments.	- Casual users *Moderate-Low.* Telemetry logs require specific tools and configurations to read and interpret the data. Because the logs are structured for monitoring rather than replay, additional setup or processing is needed to reconstruct a session. This makes it harder to replay logs across different systems without the same environment and tooling in place.
Usability	- Casual users *High.* Uses simple flags with no additional setup required, making it straightforward and predictable for casual users.	- Casual users *Moderate.* Uses the same flags, but depends on telemetry being correctly enabled and configured. This adds hidden complexity, making it less intuitive and harder to troubleshoot for non-technical users
Perfor-m ance	- Software developers, Casual users *Moderate.* Writes each event immediately to disk, which can add overhead in long sessions. Extra processing per event may slow down performance.	- Software developers, Casual users *Moderate-High.* Reuses existing telemetry pipeline, which is efficient. Some extra processing is needed to filter and reformat the data into a replay-ready NDJSON file, which can add overhead.

Based on the SAAM analysis, we determined that Approach A is the preferred solution. This approach provides higher accuracy, better usability, and more reliable replay support compared to Approach B. Although Approach A introduces slightly more I/O overhead and requires careful handling of sensitive session data, it produces logs that are easier to share and replay. It also maintains acceptable scalability and performance. In contrast, Approach B's reliance on the telemetry pipeline and exporter configuration makes replay less predictable and harder for casual users to use. Overall, Approach A aligns best with the NFRs with minimal tradeoffs.

5. Effects of Enhancement

5.1 High- and Low-Level Conceptual Architecture

As Figure 1 shows, the addition of the replay feature does not significantly change the overall layered architecture of Gemini CLI. The application layer continues to handle user input through the CLI and InputHandler, while the server layer manages core logic and communication with the LLM API and security components. The service layer remains responsible for tool

execution. The enhancement introduces three new components: SessionEventBus in the server layer, and SessionLogger and ReplayRunner in the service layer. The Core component now emits session events to the SessionEventBus, which are then collected by the SessionLogger to record sessions and then by the ReplayRunner to replay them. These create new dependencies between Core, the SessionEventBus, the SessionLogger, and ReplayRunner.

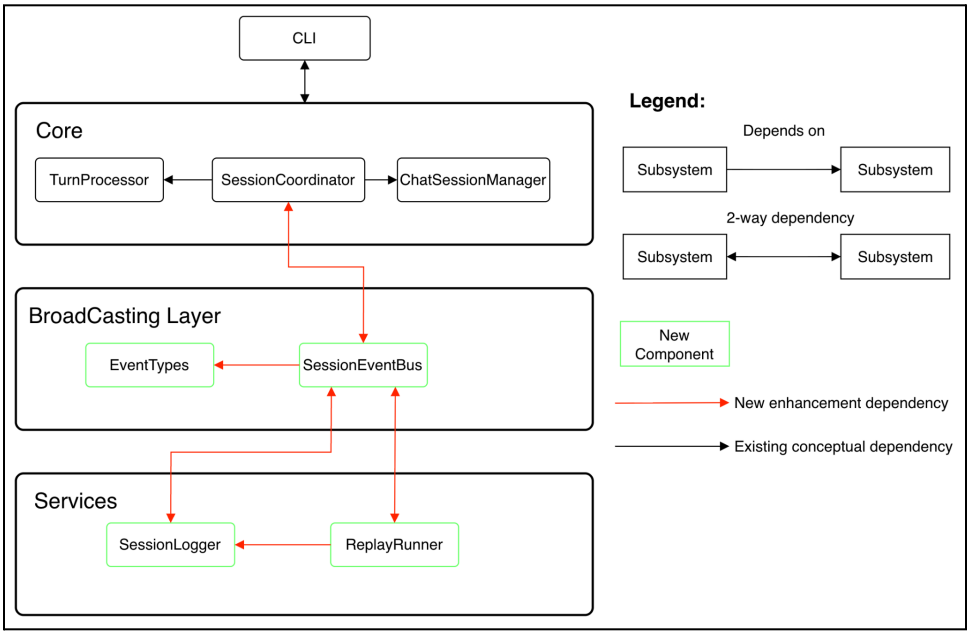

Figure 3. Effect of Enhancement on Low-Level Conceptual Architecture

The architecture introduces 4 new components across a new and existing layer. The broadcasting layer adds SessionEventBus and EventTypes to handle event routing. SessionCoordinator in core depends on SessionEventBus to publish session events. The Services layer introduces SessionLogger and ReplayRunner, which both subscribe to SessionEventBus to enable session recording and replay.

5.2 Maintainability, Evolvability, Testability, and Performance

Maintainability

The overall maintainability of Gemini CLI will decrease a bit due to the introduction of new components (SessionEventBus, SessionLogger, ReplayRunner). These components introduce additional logic that must be maintained, increasing the overall complexity. This higher complexity is likely to increase the likelihood of bugs and the effort needed to understand and update the system. However, because the architecture follows a modular design, the impact is not severe. As a result, while there is more code to maintain, the clear separation of concerns helps preserve long term maintainability.

Evolvability

The overall evolvability of Gemini CLI will improve because of the introduction of the SessionEventBus. This channel, through which all session events are emitted, provides a strong foundation for new enhancements such as replaying only failed tool calls for debugging or

triggering automated suggestions when certain events occur. By handling these features through subscribers, developers can extend functionality without changing the core logic.

Testability

The testability of Gemini CLI will improve with this enhancement. The publish-subscribe architecture allows individual components such as the SessionLogger and ReplayRunner to be tested in isolation by simulating events on the SessionEventBus. This makes it easier to write unit tests for specific behaviours (for example verifying that a user prompt event is correctly logged). Also, replay files provide a deterministic way to reproduce the sequence of events in a session which is useful for debugging. However, the outputs from the LLM may still vary between runs.

Performance

The performance of Gemini CLI may experience a slight decrease in performance when the recording or replay features are active. During recording, the SessionLogger writes each event to disk, which introduces some overhead, especially for long sessions. During replay, the system processes the recorded events in addition to its normal tasks, which also adds processing time. However, both of these features are opt-in, so when the CLI is used normally (without –record and –replay), there is no performance impact.

6. Interactions with Other Features

The proposed session replay feature would provide structured logging and playback of Gemini CLI sessions, including user prompts, model responses, and tool interactions. When invoked through the --record flag, InputHandler parses the flag and instantiates SessionLogger, attaching it to the current session before any prompts are sent. A new SessionEventBus inside packages/core acts as a publisher in a pub-sub architecture, emitting meaningful session events such as user turns, tool requests, model responses, and tool results as they occur. SessionLogger subscribes to these broadcasts and formats each incoming event into a structured JSON log entry, appending to the log file asynchronously so that the existing interaction loop in core is never blocked or interrupted. Because SessionEventBus sits alongside the existing control flow as an observer rather than inside it, no changes are required to the core interaction loop, which already manages session state independently.

As the session progresses, SessionEventBus captures a complete picture of the session by emitting events from the orchestration layer as a side effect of its normal operation. When GeminiClient sends a request and receives a streaming response, SessionEventBus emits both the outgoing prompt and assembled response as discrete events, including token counts, model ID, and response duration. When the Tool Executor processes a tool invocation, it emits the tool name, arguments, and the user's approval decision, followed by the tool result or error once execution completes. State Management contributes session-level metadata such as session ID and model version to the log header at initialization, keeping the log self-contained without requiring additional lookups at replay time.

When the session ends, SessionLogger listens for a completion signal from Core, finalizes the log file with a session summary, and passes the collected JSON file up to Packages/CLI for display as a brief end-of-session summary. Outside of this, the CLI behaves identically to a normal session, meaning the feature is entirely transparent when not explicitly

activated. When the user later invokes --replay <file>, InputHandler routes the request to ReplayRunner instead of starting a live session. ReplayRunner reads the JSON log and re-feeds the recorded events back through SessionEventBus into Core, allowing the session to be reviewed without contacting the Gemini API. For cross-model comparison, ReplayRunner re-submits the original user prompts through GeminiClient to a different configured model, and the CLI displays both responses side by side. SessionLogger and ReplayRunner are opt-in subscribers that only attach when the --record or --replay flags are passed, meaning the entire mechanism is invisible and zero-cost in a normal interactive session.

7. Impacted Directories and Files

In Gemini CLI, we would first create a new directory in "*packages/core/src/"* called "*session-event-bus/"*. To this directory, we would add the files *"sessionEventBus.ts"* as the broadcaster component for our new feature, and *"types.ts"* to define record/replay event and message schema. To connect this broadcasting component to the current system, we would modify *"packages/core/src/core/turn.ts"* and *"packages/core/src/core/client.ts"*, as these files already exist in the system layer responsible for session activity, and thus would be important to connect to *"sessionEventBus.ts"* to ensure the broadcasting component broadcasts all relevant events during execution.

Next, we would create a new file, called *"sessionLogger.ts"* in *"packages/core/src/services/"*. This file would implement the SessionLogger subscriber component, which listens to the SessionEventBus component and writes each emitted event to an NDJSON log file on disk. After that, we would create another file called *"replayRunner.ts"* in *"packages/core/src/services/"*. This would implement the ReplayRunner component, reading previously recorded logs made by the SessionLogger, and replaying the recorded session flow. To allow access to this feature, we must modify the *"packages/cli/src/config/config.ts"*, *"packages/cli/src/gemini.tsx"*, and *"packages/cli/src/nonInteractiveCliAgentSession.ts"* files to allow the CLI to parse the --record and --replay flags, and choose between logging or replay actions.

Finally, we would modify *"packages/core/src/config/config.ts"* so that our new feature is able to recognize and handle the --record and --replay commands when passed to the core from the CLI. A test suite would also be added for each new component. The main new test files would be *"packages/core/src/session-event-bus/sessionEventBus.test.ts", "packages/core/src/services/sessionLogger.test.ts"*, and *"packages/core/src/services/replayRunner.test.ts"*. In addition, existing tests for all modified components would also need to be updated or expanded to reflect the new record/replay functionality.

Overall, the replay feature would add four new files: *"sessionLogger.ts", "replayRunner.ts", "sessionEventBus.ts",* and *"types.ts"*, while primarily affecting existing directories *"packages/core/src/services/"*, *"packages/core/src/core/", "packages/core/src/config/"*, and *"packages/cli/src/"*. Some additional files would likely need modification as well, but the files listed here are the most crucial to the implementation and overall functionality of the feature.

8. Use Cases

8.1 Use Case 1

A User enables replay mode, sends a prompt, gets a response from a model, then uses replay mode to send a prompt to a different model, and compares the output of both models.

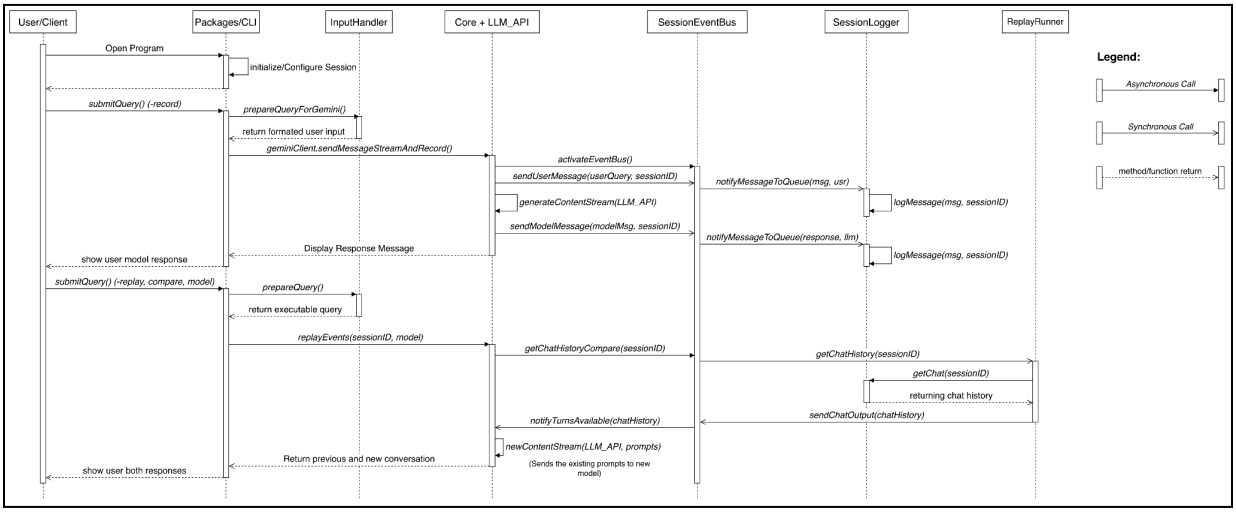

Figure 4: Sequence Diagram For User Case 1

As shown in Figure 4, the process begins when the user opens the Gemini CLI application and submits a prompt using the --record command. The CLI prepares and formats this input, ensuring that Core understands the input. It is then forwarded to Core from CLI, which then sends the prompt to the LLM API for a response, and SessionEventBus to announce the prompt, and allow SessionLogger to record it. The model then finishes generating its response, sends it back to Core, which forwards it to SessionEventBus to announce the response and allow SessionLogger to record it. After the model response is returned to the user, the user uses the --replay command and the new LLM API in their next prompt. This prompt is forwarded to Core from CLI, which sends a message to SessionEventBus, broadcasting the request. ReplayRunner calls SessionLogger, getting the chat history, sends it to SessionEventBus, which announces it. Core takes this announcement, sends a request to the new LLM with the existing prompts, and returns the result of the old and new chats back to the user through CLI.

8.2 Use Case 2

A user enables the --replay feature on a recorded session to review the session turn by turn in order to identify a bug introduced by the LLM at some point earlier.

As shown in Figure 5, the process begins when a user submits a replay request using the --replay command. The CLI forwards the request to InputHandler, which formats and configures the request for Core. CLI then sends this request to Core, which forwards it to SessionEventBus, which broadcasts the replay request so that ReplayRunner can retrieve the recorded session in question from SessionLogger. ReplayRunner then sends this chat back through SessionEventBus,

which notifies Core with a broadcast containing the replayData. Core then returns each turn of the chat through CLI, which displays the turn to the developer one at a time. Once the review is complete, the replay session ends and the application proceeds as normal.

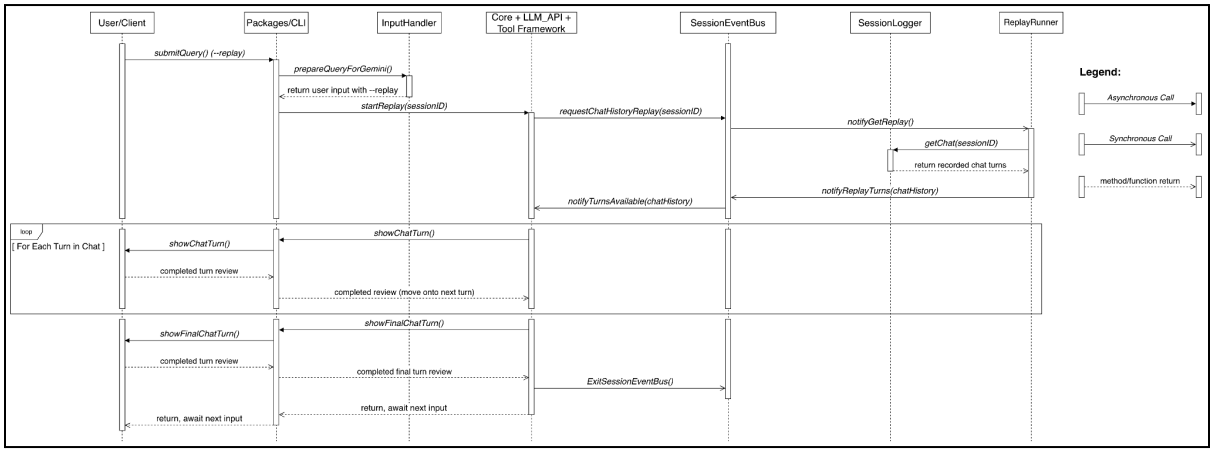

Figure 5: Sequence Diagram For User Case 2

9. Potential Risks from Enhancement

One potential risk for this enhancement is the possibility of incomplete or inaccurate session recording. Since Approach A relies on the publish-subscribe event system where different components emit events during execution, there is a chance that some important events may not be captured or may be emitted in the wrong order. This would affect the correctness of the replay feature and could lead to sessions that do not accurately reflect what originally happened. As a result, users may lose trust in the replay output, especially when using it for debugging or comparing model behaviour.

Another risk is the performance overhead introduced by the event broadcasting system. Even though SessionLogger is designed as an optional component that is only enabled when recording is turned on, continuously emitting and handling events during a session may still introduce latency, especially in long or tool-heavy sessions. If event handling is not implemented efficiently or becomes blocking, it could slow down the CLI experience and negatively impact responsiveness for users.

There are also storage and scalability concerns. Recording every session in detail can produce large JSON log files, particularly for long sessions involving multiple tool calls and responses. Over time, this could lead to high disk usage and make it more difficult to manage or transfer replay files. This risk becomes more significant when users record multiple sessions at once or run extended comparisons between models.

Finally, there are potential maintainability risks due to the added complexity in the core architecture. Introducing a new event bus and multiple subscribers increases the number of moving parts in the system. If it is not carefully designed, this could make debugging internal issues more difficult for developers, especially when trying to trace whether an issue comes from the core logic, the event system, or the logging/replay components.

10. Plans for Testing

The session replay feature introduces new components, such as the SessionEventBus, Session Logger, and ReplayRunner, which interact with the existing core and CLI architecture. These interactions must be thoroughly tested to ensure that sessions are recorded accurately and replayed correctly without affecting normal CLI performance.

To test the interaction between the session management logic in core and the SessionEventBus, developers can simulate a standard CLI session with predefined inputs and tool calls. During this test, assertions can be made to verify that all expected events are emitted in the correct order. This ensures that the event bus is capturing a complete and consistent sequence of session activities.

Testing the SessionLogger component involves verifying that emitted events are correctly formatted and written to the JSON log file. Developers can use mock events or run controlled sessions and then inspect the generated log file to ensure that all necessary fields, such as event type and timestamps, are present and accurate. Additional tests should confirm that logging does not interfere with the responsiveness of the CLI.

Replay functionality can be tested by using the ReplayRunner with previously recorded session files. Developers can compare the replayed session against the original execution to verify that the sequence of events and outputs match as expected. For model comparison, tests can be conducted by replaying the same session with a different model and checking that the system correctly displays both outputs for evaluation.

It is also important to test edge cases, such as long sessions, sessions with many tool calls, or incomplete log files. These tests ensure that the system handles large amounts of data and potential errors gracefully. For example, developers can simulate corrupted or partially written log files to verify that ReplayRunner can detect issues and fail safely without crashing the system.

Finally, testing should confirm that the feature works correctly when triggered through the CLI using the --record and --replay flags. This includes verifying that SessionLogger is only activated when enabled and that ReplayRunner properly overrides the normal execution flow during replay.

11. Lessons Learned and Limitations

One of the main limitations of this report is that we did not have enough time to fully understand how the system would be affected by our proposed feature. While we were able to identify the key components that would need to change and describe how they would interact, a deeper analysis of the source code would likely reveal additional dependencies and edge cases that our proposal does not account for. As a result, some of the architectural decisions described in this report are based on our best understanding of the system rather than a complete and verified implementation-level analysis.

This limitation also meant that our enhanced architectures may not fully reflect the reality of an actual implementation. There would likely be absences and divergences between our proposal and a concrete architecture that included the new feature, particularly around the

integration of SessionEventBus into the existing core server layer, which would require more careful source-level investigation than was possible within the scope of this assignment.

Overall, a major lesson learned from this project was the importance of understanding an existing system before trying to extend it. Working on this feature showed us how even a small addition can affect many parts of the architecture in ways that are not immediately obvious. This made us appreciate the value of the structured architectural analysis methods we practiced throughout this course.

12. Conclusion

In conclusion, our report proposed a new session replay and recording feature for Gemini CLI, providing an in-depth analysis of its architectural considerations and implications. We presented two approaches to implement this feature: Approach A, which introduces a publish-subscribe style event system through a central SessionEventBus, and Approach B, which extends the existing OpenTelemetry pipeline with a replay-oriented exporter. After conducting a SAAM analysis evaluating both approaches against key stakeholders and non-functional requirements, we determined that Approach A was the overall best implementation due to its superior maintainability, testability, and accuracy.

Approach A was then explored further in relation to its impact on the high- and low-level conceptual architectures, as well as its interactions with existing features such as the core orchestration layer, InputHandler, and the Tool Execution Framework. We demonstrated through two use cases how the feature flows through the proposed architectural modifications and identified the directories and files that would be impacted by the implementation. Additionally, we assessed the potential risks of the enhancement, including performance overhead, storage concerns, and maintainability challenges, and outlined a testing plan to address these risks.

Overall, this project gave us a deeper appreciation for the complexity involved in introducing new features to an existing system. Building on the architectural knowledge gained from our conceptual and concrete architecture reports, we were able to propose a feature that extends Gemini CLI in a meaningful way while keeping disruption to the existing architecture minimal.

13. Data Dictionary

SessionEventBus: A new publish-subscribe broadcast component added to the server layer of packages/core that emits meaningful session events to its subscribers.

NDJSON: Newline-Delimited JavaScript Object Notation. A file format where each line is a valid JSON object, allowing large log files to be read and processed line by line without loading the entire file into memory.

Event: A discrete, meaningful action emitted by the SessionEventBus during a session, such as a user turn, model response, tool request, tool result, or approval decision.

Replay: The act of reconstructing and re-running a previously recorded session using a log file, either for review or for comparison against a different model.

Flag: A command-line argument passed by the user to modify the behaviour of Gemini CLI, such as --record to activate session logging or --replay to activate session playback.
SAAM: Software Architecture Analysis Method. A method used to evaluate a system's architecture against stakeholder non-functional requirements.
NFR: Non-Functional Requirement. A requirement that describes how a system performs a function, rather than what function it performs.

14. Naming Conventions

CLI: Command Line Interface
Core: Backend service logic/business logic layer
JSON: JavaScript Object Notation
NDJSON: Newline-Delimited JavaScript Object Notation

15. AI Collaboration Report

For A3, our AI teammate was Perplexity (March 2026 version). We selected this tool because the assignment required architectural analysis supported by documentation, and Perplexity is particularly strong at retrieving source-linked technical information. Unlike a single-model assistant, Perplexity operates as an AI agent that can automatically select and route queries to different models such as GPT, Claude, or Gemini, depending on the task. This flexibility made it well-suited for our workflow, since different tasks such as retrieval, summarization, and critique benefited from different model strengths.

Tasks Assigned to the AI Teammate

Locate and summarize existing architectural patterns relevant to the proposed replay feature, particularly publish-subscribe and telemetry-based approaches. Rationale: This task involved searching through large amounts of architectural documentation and extracting relevant source-backed statements, which is well-suited to an AI optimized for fast retrieval.

Summarize lengthy technical documentation on OpenTelemetry and NDJSON into concise architecture-relevant takeaways for Approach B. Rationale: The sources were dense and technical, so using AI helped quickly produce structured summaries that we could then verify against the originals.

Suggest refinements to our SAAM analysis by identifying missing stakeholders and NFRs. Rationale: This task involved checking how our stakeholder list and NFR coverage matched a broader set of concerns, which the AI could do quickly to support our review.

Interaction Workflow

Our group used a collaborative workflow: each member drafted prompts for their section and shared both the prompts and the AI outputs with the group. Before sending prompts, we first discussed what we wanted to learn from the AI and gathered local context such as our current diagrams, assumptions, and assignment requirements. After receiving responses, we reviewed

them together, decided what was useful, and edited or discarded anything unclear or unsupported.

One of our more complex prompts asked the AI to act like a software architecture TA and critique our proposed Approach A and Approach B designs for the replay feature. We provided context such as our component descriptions, the subsystems we were analyzing, and our SAAM table, and requested a clear structured response identifying weaknesses in our NFR evaluations and missing architectural considerations. Initially responses were too general, so we refined the prompt by adding more specific details about our system and requiring the AI to ground its feedback in concrete architectural reasoning. These refinements improved the relevance and quality of the output.

Validation Procedures

All AI outputs were treated as drafts. We verified factual claims by checking primary sources, cross-referencing summaries with original documentation, and edited all text for clarity and accuracy. Any claim that could not be validated was removed or clearly labelled as an assumption.

Estimated Contribution

We estimate that the AI contributed approximately 12% of the final deliverable:

- 5% for summarizing technical documentation and locating relevant architectural patterns
- 4% for suggesting refinements to the SAAM analysis and identifying missing NFR considerations
- 3% for research support on publish-subscribe and telemetry architectures

All architectural decisions, interpretations, and diagrams remained human-generated.

Reflection on Team Dynamics

The AI improved efficiency in drafting, research, and brainstorming, but it also introduced additional verification work. It helped identify missing assumptions and improved clarity through rewriting support. However, some responses sounded confident even when partially inferred, so our group adopted a strict rule that factual claims required source validation. Overall, we learned that effective AI collaboration requires clear prompts, structured outputs, and consistent verification.

16. References

Observability with OpenTelemetry. Gemini CLI. (n.d.-b).

https://geminicli.com/docs/cli/telemetry/